

dogmatch_ai

Relazione Strutturale

Servizio AI - FastAPI + GPT-4o + Sentence Transformers
Maggio 2026

Stack	Python 3.11, FastAPI 0.136, Uvicorn, aiomysql
AI Model	GPT-4o (OpenAI SDK 2.36.0)
Embeddings	sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (384-dim)
Database	MySQL con pool asincrono aiomysql
Autenticazione	JWT HS256 (opzionale - disabilitabile)
Tool disponibili	14 tool per GPT-4o (search, memory, booking, moderation...)
Rate limiting	10 req/60s per utente - in-memory
Notifiche proattive	Background loop ogni 5 minuti
Web search	Tavily API (opzionale)
Testing	pytest + pytest-asyncio, 6 file di test

1. Visione generale

dogmatch_ai e' il cervello del chatbot SnoutBot. E' un microservizio Python/FastAPI che espone due endpoint REST (/chat e /chat/stream) e implementa un loop di tool-calling orchestrato con GPT-4o. I client non lo contattano mai direttamente: le richieste arrivano sempre dal backend_node (Node.js) che costruisce il system prompt e funge da proxy autenticato.

Flusso generale

```

backend_node:3000
  POST /api/chat/bot (JWT verificato, context costruito da MySQL)
    |
    v
dogmatch_ai:8000
  POST /chat
    |
    v
  orchestrator.get_ai_response()
  +--> _load_user_memory()      (ai_memoria)
  +--> _load_unread_notifications() (ai_notifiche)
  +--> _rag_context()          (se ricerca cani: pre-fetch top-5)
  +--> _build_messages()       (system prompt + history)
  +--> GPT-4o tool-calling loop (max 5 iter)
        +--> execute_tool() -> 14 tools disponibili
        +--> ritorna {risposta: string}
  
```

Dipendenze principali (requirements.txt)

Pacchetto	Versione	Ruolo
fastapi	0.136.1	Framework web asincrono
uvicorn	0.46.0	ASGI server
openai	2.36.0	Client SDK GPT-4o
sentence-transformers	5.4.1	Embedding semantici multilingua
torch	2.11.0	Dipendenza di sentence-transformers
scikit-learn	1.8.0	TF-IDF + cosine similarity (fallback)
aiomysql	0.3.2	Driver MySQL asincrono
pydantic / pydantic-settings	2.13.4 / 2.14.1	Validazione dati + config .env
PyJWT	2.12.1	Verifica token JWT
httpx	0.28.1	Client HTTP asincrono (Tavily)
python-dotenv	1.2.2	Caricamento .env
pytest / pytest-asyncio	9.0.3 / 1.3.0	Test framework

2. Configurazione (config.py + .env)

Variabile .env	Default	Obbligatoria	Descrizione
OPENAI_API_KEY	--	SI	Chiave API OpenAI
DB_HOST	localhost	no	Host MySQL
DB_PORT	3306	no	Porta MySQL
DB_NAME	dogmatch_db	no	Nome database
DB_USER	root	no	Utente DB

DB_PASSWORD	root	no	Password DB
MODEL	gpt-4o	no	Modello OpenAI
MAX_TOKENS	1024	no	Token max per risposta
MAX_TOOL_ITERATIONS	5	no	Max iterazioni tool-calling
JWT_SECRET	None	no	Se assente: auth disabilitata
JWT_ALGORITHM	HS256	no	Algoritmo JWT
ALLOWED_ORIGINS	localhost:3000	no	CORS origins (separati da virgola)
TAVILY_API_KEY	None	no	Web search (opzionale)

Attenzione: Se `JWT_SECRET` non e' impostato, l'autenticazione e' completamente disabilitata. In produzione impostare sempre `JWT_SECRET` e legare `dogmatch_ai` a `127.0.0.1`.

3. Entry point - main.py

Endpoint REST

Endpoint	Auth	Rate limit	Descrizione
GET /health	no	no	Health check: {status, service, model}
POST /chat	JWT opz.	10/60s per user	Chat sincrona -> {risposta: string}
POST /chat/stream	JWT opz.	10/60s per user	Chat SSE (token progressivi)
POST /admin/knowledge	JWT req.	no	Inserisce/aggiorna voce KB nel DB

Middleware CORS

Parametro	Valore
allow_origins	Da config.settings.cors_origins (split di ALLOWED_ORIGINS)
allow_methods	GET, POST
allow_headers	*

Lifespan - Startup

1. Crea pool MySQL asincrono (minsize=2, maxsize=10, charset=utf8mb4)
2. Crea tabelle se non esistono:
 - ai_memoria (user_id, chiave, valore, aggiornato_at)
 - ai_knowledge_base (chiave PK, titolo, contenuto, tags JSON)
 - ai_notifiche (id, user_id, cane_id, messaggio, letta, creata_at)
 - aree_cani (id, nome, citta, provincia, separazione_taglie)
 - prenotazioni_area (codice, user_id, area_id, data_ora, stato ENUM)
3. Aggiunge colonne mancanti a 'cani':
 - creato_at DATETIME
 - embedding MEDIUMBLOB
 - embedding_hash VARCHAR(64)
4. Carica knowledge_base.json nel DB (solo se tabella vuota)
5. Seed 6 aree cani certificate (solo se tabella vuota)
6. Avvia background task: notification_loop() (ogni 5 minuti)

Lifespan - Shutdown

1. Cancella notification_loop (asyncio task)
2. Chiude pool MySQL

4. Orchestratore AI (orchestrator.py)

L'orchestratore e' il cuore del servizio. Gestisce il ciclo completo: rilevamento intent, pre-fetch RAG, costruzione del system prompt e il loop di tool-calling con GPT-4o.

4.1 Intent Detection (_is_dog_search)

Prima di costruire il prompt, l'orchestratore decide se la richiesta e' una ricerca di cani. Usa due strategie in ordine di priorita':

Strategia	Quando usata	Soglia	Dettagli
Embedding-based	sentence-transformers disponibile	cosine >= 0.35	Confronta con centroide di 10 frasi prototipo (384-dim)
Keyword fallback	sentence-transformers assente	match esatto stem	24 stem: cerc, trov, match, compagn, mite, energic...

4.2 RAG Pre-fetch (_rag_context)

Se il messaggio e' una ricerca cani, carica in anticipo i top-5 risultati e li inserisce nel system prompt. Questo riduce il numero di tool-call necessari durante la conversazione.

Se `_is_dog_search(message) == True`:

1. Carica profilo cane utente corrente
2. Arricchisce query: `message + dog_profile`
3. Chiama `search_dogs_semantic()` -> top-5 risultati
4. Ritorna risultati formattati per system prompt

4.3 System Prompt (_build_messages)

Il system prompt e' composto in questo ordine:

1. `request.context` (dal backend_node: profilo utente + cani dal DB)
2. `BOT_IDENTITY` (SnoutBot, assistente AI di DogMatch)
3. Nome utente (se presente in context)
4. Memory storage (fatti e promemoria da `ai_memoria`)
5. Notifiche non lette (menzione proattiva per ogni notifica non letta)
6. RAG context (cani pre-caricati, se ricerca cani)
- poi history ---
7. History completo (messaggi precedenti `role:user/assistant`)
8. Messaggio utente corrente

4.4 Tool-Calling Loop (get_ai_response)

Per max `MAX_TOOL_ITERATIONS` (default=5) iterazioni:

```
risposta = client.chat.completions.create(
    model=settings.model,
    messages=messages,
    tools=TOOLS,
    tool_choice='auto',
    max_tokens=settings.max_tokens
)

se finish_reason == 'stop':
    return message.content

se finish_reason == 'tool_calls':
    per ogni tool_call in message.tool_calls:
```

```
        result = execute_tool(name, arguments, user_id)
        aggiungi {role:'tool', tool_call_id:id, content:result}
        continua iterazione

se finish_reason == 'length':
    return risposta troncata

se iterazioni esaurite:
    return messaggio errore
```

4.5 Streaming Mode (get_ai_response_stream)

Identico al loop sincrono ma con stream=True. Ogni delta.content viene emesso come SSE: 'data: {"token": content}\n\n'. Al termine emette 'data: [DONE]\n\n'.

5. Sistema dei Tool (14 tool per GPT-4o)

Panoramica tool disponibili

Tool	Parametri principali	Funzione
search_dogs_semantic	query, taglia, sesso, razza, eta_max, disponibile_rig	Ricerca semantica cani con embedding
analyze_user_profile	user_id	Completeness profilo + suggerimenti
get_knowledge_base_info	topic	Recupera info da knowledge base
update_user_profile	user_id, field, value	Aggiorna bio/provincia/regione/telefono
save_memory	chiave, valore	Salva fatto persistente per utente
save_reminder	id, testo	Salva promemoria (prefisso todo_)
clear_reminder	id	Rimuove promemoria
search_web	query	Ricerca Tavily (opzionale)
get_activity_stats	(nessuno)	Match attivi, like ricevuti, messaggi
get_available_slots	data, area_id (opz.)	Slot liberi parchi cani
book_area_slot	area_id, data_ora	Prenota slot (codice DM-XXXXXXX)
get_user_bookings	(nessuno)	Prenotazioni attive utente
cancel_booking	codice	Cancella prenotazione per codice
moderation_flag	content, reason	Segnalazione audit log JSONL

5.1 search_dogs_semantic - Dettaglio

E' il tool piu' complesso. Usa sentence-transformers per trovare i cani piu' compatibili con la query dell'utente tramite cosine similarity.

Modello: paraphrase-multilingual-MiniLM-L12-v2 (384 dimensioni)

Pipeline:

1. SELECT cani JOIN utenti (con filtri opzionali applicati via SQL)
2. _build_dog_text(): razza + taglia + sesso + eta + descrizione + info_sanitarie
3. Embedding:
 - Separa cani cached (hash match) da quelli da computare
 - Batch encode solo non-cached
 - Save batch async in DB (MEDIUMBLOB)
 Fallback TF-IDF se sentence-transformers non disponibile
4. Ranking: cosine similarity, soglia minima 0.20, top-5
5. Output: markdown con % affinita' per ogni cane

Cache LRU in-memory:

- Max 500 entry per dog_id
- Eviction FIFO quando piena
- Invalidazione su hash mismatch (testo cane modificato)

5.2 Booking System - Dettaglio

Regola	Valore
Orario apertura parchi	08:00 - 19:00 (slot da 1 ora)
Anticipo massimo prenotazione	30 giorni
Formato codice prenotazione	DM-{8 char alfanumerici}

Stato prenotazione	ENUM: attiva cancellata
Slot bloccati	No slot passati, no duplicati stesso utente stesso orario

6 Aree Cani Certificate (seed)

Nome	Citta	Sep. taglie
Area Cani Parco Sempione	Milano	Si
Dog Park Navigli	Milano	No
Area Cani Villa Borghese	Roma	Si
Dog Park Valentino	Torino	No
Area Cani Parco Virgiliano	Napoli	No
Dog Area Cascine	Firenze	Si

5.3 Memory System - Dettaglio

Tabella: ai_memoria (user_id, chiave, valore, aggiornato_at)

Tipi di record:

- Fatti: chiave = 'nome_cane', 'obiettivo', 'razza_preferita'...
- Promemoria: chiave = 'todo_foto_profilo', 'todo_vaccinazioni'...

load_user_memory(user_id):

SELECT * WHERE user_id = X

Separa fatti (no prefisso todo_) da promemoria (prefisso todo_)

Ritorna markdown formattato inserito nel system prompt

save_memory(chiave, valore): INSERT ... ON DUPLICATE KEY UPDATE

save_reminder(id, testo): salva come chiave 'todo_{id}'

clear_reminder(id): DELETE WHERE chiave = 'todo_{id}'

5.4 Moderation - Dettaglio

Le segnalazioni sono append-only su file JSONL (moderation_log.jsonl). Non bloccano l'utente immediatamente: il team di moderazione decide le azioni.

Record salvato:

```
{
  flag_id: UUID[:8],
  timestamp: ISO8601,
  user_id: string,
  reason: string,
  content_preview: content[:200]
}
```


6. Notifiche Proattive (background.py + notifications.py)

Un loop in background scansiona ogni 5 minuti i nuovi cani registrati e genera notifiche personalizzate per gli utenti con preferenze compatibili.

Pipeline notifiche

```
notification_loop() -> ogni 5 minuti:

run_notification_scan(since=last_scan_timestamp)
|
+-- _new_dogs_since(since)
|   SELECT cani JOIN utenti WHERE creato_at > since
|
+-- _users_with_preferences()
|   SELECT DISTINCT user_id FROM ai_memoria WHERE chiave NOT LIKE 'todo_%'
|
+-- Per ogni (nuovo_cane, utente):
|   query = _user_preference_text(user_id)
|           (concatena tutti i valori memoria)
|   similarity = cosine(embed(query), embed(cane_text))
|   if similarity >= 0.45:
|       _save_notification(user_id, cane_id, messaggio)
|       (INSERT IGNORE -> no duplicati)
|
+-- Aggiorna last_scan_timestamp
```

Parametro	Valore
Frequenza scan	Ogni 300 secondi (5 minuti)
Soglia match	0.45 cosine similarity
Anti-duplicati	INSERT IGNORE su UNIQUE KEY (user_id, cane_id)
Lettura notifiche	Caricate in ogni request /chat, marcate lette=1 subito
Integrazione prompt	Notifiche non lette inserite nel system prompt (punto 5 ordine)

7. Database (database.py)

Schema tabelle create dal servizio AI

Tabella	Colonne principali	Scopo
ai_memoria	user_id, chiave, valore, aggiornato_at	Fatti e promemoria per utente
ai_knowledge_base	chiave PK, titolo, contenuto, tags JSON	KB del dominio cani
ai_notifiche	id, user_id, cane_id, messaggio, letta, creata_at	Notifiche proattive
aree_cani	id, nome, citta, provincia, separazione_taglie	Parchi certificati (seed)
prenotazioni_area	codice, user_id, area_id, data_ora, stato ENUM	Slot prenotazioni

Colonne aggiunte a tabelle esistenti (migration on startup)

Tabella	Colonna aggiunta	Tipo
cani	creato_at	DATETIME - trigger notifiche proattive
cani	embedding	MEDIUMBLOB - vettore 384-dim serializzato
cani	embedding_hash	VARCHAR(64) - SHA256 del testo cane

Pool di connessioni

Parametro	Valore
Driver	aiomysql (asincrono)
minsize	2 connessioni
maxsize	10 connessioni
charset	utf8mb4
autocommit	True
cursor	DictCursor (righe come dizionari)

8. Modelli Pydantic (models.py)

```

class HistoryMessage(BaseModel):
    role: str          # 'user' | 'assistant'
    content: str

class ChatRequest(BaseModel):
    message: str = Field(..., max_length=500)
    user_id: str
    context: str       # system prompt pre-costruito da backend_node
    history: List[HistoryMessage] = []

class ChatResponse(BaseModel):
    risposta: str

```

9. Rate Limiting (rate_limit.py)

Parametro	Valore
Finestra temporale	60 secondi

Limite per utente	10 richieste
Implementazione	In-memory defaultdict con lista di timestamp
Pulizia	Timestamp < (now - 60s) rimossi ad ogni check
Errore	HTTP 429 con messaggio dettagliato
Isolamento	Ogni user_id ha il proprio contatore indipendente

10. Knowledge Base (knowledge_base.json + knowledge.py)

Struttura del file JSON

```
{
  "chiave_univoca": {
    "title": "Titolo voce",
    "content": "Contenuto in markdown...",
    "tags": ["tag1", "tag2"]
  },
  ...
}
```

42 voci disponibili - Argomenti principali

Chiave	Argomento
registrazione_onboarding	Step registrazione, validazioni, errori comuni
login_accesso	Login, recupero password, OTP, blocco account
discovery_filtri	Feed discovery, filtri (intento, distanza, razza)
sistema_match	Like/dislike, match reciproco, notifiche match
accoppiamento_riproduzione	Requisiti, raccomandazioni sanitarie, badge
profilo_badge_verifica	Stati badge, completeness %, campi modificabili
ban_ricorso_ticket	Motivi ban, apertura ricorso, stati ticket
chat_messaggi	Interfaccia chat, limite 1000 char, sicurezza
notifiche	Tipi notifica, gestione, impostazioni
razze_disponibili	20+ razze popolari su DogMatch
taglie_peso	Piccola/Media/Grande/Gigante con range kg
aree_cani_incontri	6 aree certificate, come prenotare, regole
snoutbot_guida	Cosa chiedere a SnoutBot, limiti
sicurezza_privacy	Dati personali, incontri sicuri, segnalazioni
ruoli_utente	Privato, Allevatore, Appassionato, Admin
salute_vaccinazioni	Calendario vaccinazioni, microchip, obblighi
parassiti / alimentazione	Prevenzione parassiti, dieta e nutrizione
malattie / comportamento	Malattie comuni, problemi comportamentali
addestramento / leggi	Addestramento base, leggi e normative

Algoritmo di ricerca (knowledge.py)

Relevance scoring per topic ricevuto:

```
1.00 -> Exact key match
0.95 -> Title match esatto
0.85 -> Tags match
0.40-0.60 -> Word matching + SequenceMatcher
```

```
Ritorna top-2 risultati con score > 0.15
Se nessun risultato: lista argomenti disponibili
```

11. Test Suite (tests/)

6 file di test con pytest + pytest-asyncio. asyncio_mode = auto (pytest.ini).

File	Cosa testa
test_config.py	Caricamento settings, CORS origins parsing, max_tokens/max_tool_iterations positivi
test_rate_limit.py	Permette < limite, blocca al limite (429), utenti indipendenti, reset finestra scaduta
test_search_utils.py	_build_dog_text M/F, _text_hash SHA256 64char, LRU cache hit/miss/eviction/indipendenza
test_orchestrator.py	_build_messages struttura, nome nel prompt, notifiche e RAG nel prompt, _is_dog_search keyword
test_notifications.py	Soglia 0.45, math cosine similarity, prima exec baseline, skip senza cani nuovi/preferenze
test_integration.py	GET /health, POST /chat valido/con history/payload mancante (422), rate limiting 429, KB

Info: I test di integrazione richiedono il servizio avviato su localhost:8000. I test unitari (config, rate_limit, search_utils, orchestrator, notifications) sono eseguibili offline senza DB o OpenAI.

12. Script - compute_embeddings.py

Script CLI per pre-calcolare gli embedding di tutti i cani in batch. Va eseguito una volta dopo il primo deploy (o dopo import massivi di cani) per evitare latenza al primo search.

Esecuzione: `python scripts/compute_embeddings.py`

Pipeline:

1. Carica modello paraphrase-multilingual-MiniLM-L12-v2
2. Crea pool DB
3. SELECT tutti i cani
4. Filtra cani con embedding obsoleto (hash mismatch)
5. Batch encode in gruppi di 32
6. UPDATE embedding + hash nel DB
7. Stampa progress (n cani aggiornati)

13. Schema architetturale completo

FastAPI Server (main.py) - porta 8000

```
|
+-- GET /health          -> health check
+-- POST /chat           -> orchestrator.get_ai_response()
+-- POST /chat/stream    -> orchestrator.get_ai_response_stream()
+-- POST /admin/knowledge -> upsert KB (JWT required)
|
+-- CORS middleware     -> da ALLOWED_ORIGINS
+-- Rate limiter        -> 10/60s per user_id
+-- JWT auth            -> opzionale (se JWT_SECRET impostato)
```

Orchestrator (orchestrator.py)

```
+-- _is_dog_search()     -> embedding/keyword intent detection
+-- _rag_context()       -> pre-fetch top-5 cani compatibili
+-- _build_messages()    -> system prompt + history
+-- GPT-4o loop (max 5)  -> tool-calling con 14 tool
```

14 Tool (tools/)

```
+-- search_dogs_semantic -> embedding + LRU cache + DB
+-- analyze_user_profile -> completeness + suggerimenti
+-- get_knowledge_base_info -> relevance scoring
+-- update_user_profile  -> whitelist field update
+-- save/clear_memory    -> ai_memoria table
+-- save/clear_reminder  -> ai_memoria (todo_ prefix)
+-- search_web           -> Tavily API (opzionale)
+-- get_activity_stats   -> interazioni + messaggi stats
+-- get/book/cancel slots -> prenotazioni_area table
+-- moderation_flag      -> append-only JSONL
```

Background (background.py)

```
+-- notification_loop()   -> ogni 5 minuti
    run_notification_scan()
        _new_dogs_since() -> cani con creato_at recente
        _users_prefs()    -> utenti con memoria attiva
        embedding match   -> soglia 0.45
        _save_notification() -> ai_notifiche (INSERT IGNORE)
```

Database MySQL (aiomysql pool 2-10)

```
Tabelle proprie: ai_memoria, ai_knowledge_base, ai_notifiche,
                 aree_cani, prenotazioni_area
Tabelle shared:  utenti, cani (+ embedding, embedding_hash, creato_at),
                 interazioni, messaggi
```

14. Punti di attenzione

1. AI service senza auth propria (se JWT_SECRET non impostato)

Se JWT_SECRET non e' configurato nel .env, chiunque puo' chiamare POST /chat senza token. Accettabile solo se la porta 8000 e' strettamente vincolata a 127.0.0.1 e mai esposta pubblicamente.

2. Rate limiting in-memory non distribuito

I contatori sono in memoria del processo. Con piu' istanze (scaling orizzontale) ogni istanza avrebbe un contatore separato, rendendo il limite inefficace. Per produzione usare Redis o un backend condiviso.

3. Modello sentence-transformers (~471MB) a cold start

Il modello paraphrase-multilingual-MiniLM-L12-v2 viene caricato all'avvio. Il primo avvio su macchine senza il modello scaricato richiede il download da HuggingFace Hub (~471MB). Da gestire in Dockerfile/CI.

4. Embedding persistiti come MEDIUMBLOB

I vettori 384-dim serializzati sono salvati in MySQL come MEDIUMBLOB. Per grandi cataloghi di cani (>100k) un vector store dedicato (Pinecone, Weaviate, pgvector) sarebbe piu' efficiente.

5. Notifiche proattive non scalabili con singolo loop

notification_loop() e' un asyncio task nel processo FastAPI. Con molti utenti e cani, la scansione embedding ogni 5 minuti potrebbe diventare lenta. Un worker separato (Celery, ARQ) sarebbe piu' robusto.

6. MAX_TOOL_ITERATIONS fissa a 5

Dopo 5 iterazioni il loop si interrompe e ritorna un messaggio di errore generico. Per richieste complesse (es. ricerca + booking + salva memoria) 5 iterazioni potrebbero non bastare.